

鸿蒙多端应用开发

封面信息

表格

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、d20301009205
学号	2023210589
姓名	方勋
班级	软件工程 23 级 1 班
日期	2026-05-06
组别	班组 1
项目名称	云端智能畜牧养殖管理系统
技术栈	HarmonyOS (ArkTS)

实验目的

- 1. 掌握 ArkUI 声明式开发
- 2. 学会多端响应式布局
- 3. 了解传感器数据采集
- 4. 理解分布式能力原理

实验环境

表 1. 实验环境工具

表格

工具	版本	用途
DevEco Studio	5.0+	鸿蒙应用开发
HarmonyOS SDK	5.0+	鸿蒙开发工具包
模拟器	最新版	多设备模拟测试
AI 辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 功能需求

- [✓] 天气展示（城市名称、温度、天气状况、空气质量）
- [✓] 多端适配（手机竖屏、平板横屏）
- [✓] 传感器集成（光线传感器、陀螺仪）

2. 创建鸿蒙项目

2.1 新建项目

- [✓] 打开 DevEco Studio
- [✓] 新建 Empty Ability 项目
- [✓] 选择 ArkTS 语言
- [✓] 命名为 WeatherApp

3. 天气应用开发

3.1 天气首页实现

- [✓] 实现城市名称展示
- [✓] 实现温度显示

- [✓] 实现天气状况图标
- [✓] 实现空气质量展示

typescript

运行

@Entry@Component

struct Index {

 @State cityName: string = '滁州';

 @State temperature: number = 22;

 @State weather: string = '晴';

 @State aqi: number = 45;

 @State lightValue: number = 0;

build() {

 Column() {

 Text(this.cityName)

 .fontSize(24)

 .fontWeight(FontWeight.Bold)

 .margin({ top: 50 })

 Text(this.weather === '晴' ? '☀' : '☁')

 .fontSize(80)

 .margin({ top: 20 })

 Text(`\${this.temperature}° C`)

 .fontSize(60)

 .fontWeight(FontWeight.Lighter)

 .margin({ top: 20 })

 Text(this.weather)

 .fontSize(20)

 .margin({ top: 10 })

```

Row() {
  Text('空气质量: ')
  Text(`${this.aqi} - 优`)
    .fontColor(this.aqi <= 50 ? '#00ff00' : '#ff9900')
}

.margin({ top: 30 })

Row() {
  Text('光线传感器: ')
  Text(this.lightValue.toString())
    .id('lightValue')
}

.margin({ top: 20 })
}

.width('100%')
.height('100%')
.backgroundColor('#f5f5f5')
}}

```

4. 多端适配布局

4.1 响应式布局实现

- [✓] 配置断点系统 (sm/md/lg)
- [✓] 实现手机布局 (sm 断点)
- [✓] 实现平板布局 (lg 断点)
- [✓] 测试界面适配效果

typescript

运行

```
@State currentBreakpoint: string = 'md';
```

```
aboutToAppear() {
```

```
  let deviceInfo = umpInfo.getDeviceInfo();
```

```
  let windowWidth = window.getWindow().getWindowProperties().windowWidth;
```

```
  if (windowWidth < 600) {
```

```

        this.currentBreakpoint = 'sm';
    } else if (windowWidth < 840) {
        this.currentBreakpoint = 'md';
    } else {
        this.currentBreakpoint = 'lg';
    }
}

build() {
    if (this.currentBreakpoint === 'sm') {
        Column() {
            this.phoneLayout()
        }
    } else {
        Row() {
            this.tabletLayout()
        }
    }
}

@Builder phoneLayout() {
    Column() {
        Text(this.cityName).fontSize(24)
        Text(`${this.temperature}° C`).fontSize(60)
    }
}

@Builder tabletLayout() {
    Column() {
        Text(this.cityName).fontSize(32)
        Text(`${this.temperature}° C`).fontSize(80)
    }
    .width('40%')}

```

5. 传感器集成

5.1 光线传感器开发

- [✓] 导入传感器模块
- [✓] 注册光线传感器监听

- [✓] 实现环境光线数据展示
- [✓] 取消传感器监听

typescript
运行

```
import sensor from '@ohos.sensor';

@State lightIntensity: number = 0;

aboutToAppear() {
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
        this.lightIntensity = data.light;
    });
}

aboutToDisappear() {
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
}

Text(`环境光线: ${this.lightIntensity} lux`)
    .fontSize(16)
    .margin({ top: 10 })
```

5.2 陀螺仪传感器开发

- [✓] 注册陀螺仪传感器监听
- [✓] 实现三轴数据展示
- [✓] 测试设备旋转响应

typescript
运行

```
@State rotateX: number = 0; @State rotateY: number = 0; @State rotateZ: number = 0;

aboutToAppear() {
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
        this.rotateX = data.x;
        this.rotateY = data.y;
        this.rotateZ = data.z;
    });
}

Row() {
    Column() { Text(`X轴: ${this.rotateX.toFixed(2)}°`) }
    Column() { Text(`Y轴: ${this.rotateY.toFixed(2)}°`) }
    Column() { Text(`Z轴: ${this.rotateZ.toFixed(2)}°`) }.spacing(20)
```

6. 分布式数据同步

6.1 分布式数据管理

- [✓] 实现数据存储功能
- [✓] 实现数据读取功能
- [✓] 测试跨设备同步

typescript
运行

```
import distributedData from '@ohos.data.distributedData';

const KVManager = distributedData.createKVManager({

  bundleName: 'com.example.weather',

  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION});

async function saveData(key: string, value: string) {

  const kvStore = await KVManager.getKVStore('weatherStore');

  await kvStore.put(key, value);}

async function getData(key: string) {

  const kvStore = await KVManager.getKVStore('weatherStore');

  return await kvStore.get(key);}
```

7. 测试验证

7.1 多设备模拟器测试

- [✓] 启动手机模拟器
- [✓] 启动平板模拟器
- [✓] 测试界面适配效果
- [✓] 观察传感器数据
- [✓] 记录测试结果

实验结果

验证结果

表 2. 功能验证结果

表格

功能	状态	备注
天气信息展示	[✓] 成功	
多端适配	[✓] 成功	
光线传感器	[✓] 成功	
陀螺仪传感器	[✓] 成功	
分布式数据同步	[✓] 成功	

截图记录

手机端界面

上海



20°C

雨

空气质量: 65 (良)

传感器数据

😊 光线强度: 1129 lux (非常明亮)

📱 设备旋转角度:

X轴	Y轴	Z轴
4.8°/s	-3.92°/s	-3.84°/s

刷新天气

切换城市

设备信

合肥



22°C

晴

空气质量: 45 (优)

传感器数据

😊 光线强度: 1550 lux (非常明亮)

📶 设备旋转角度:

X轴	Y轴	Z轴
-2.02°/s	-0.29°/s	0.75°/s

刷新天气

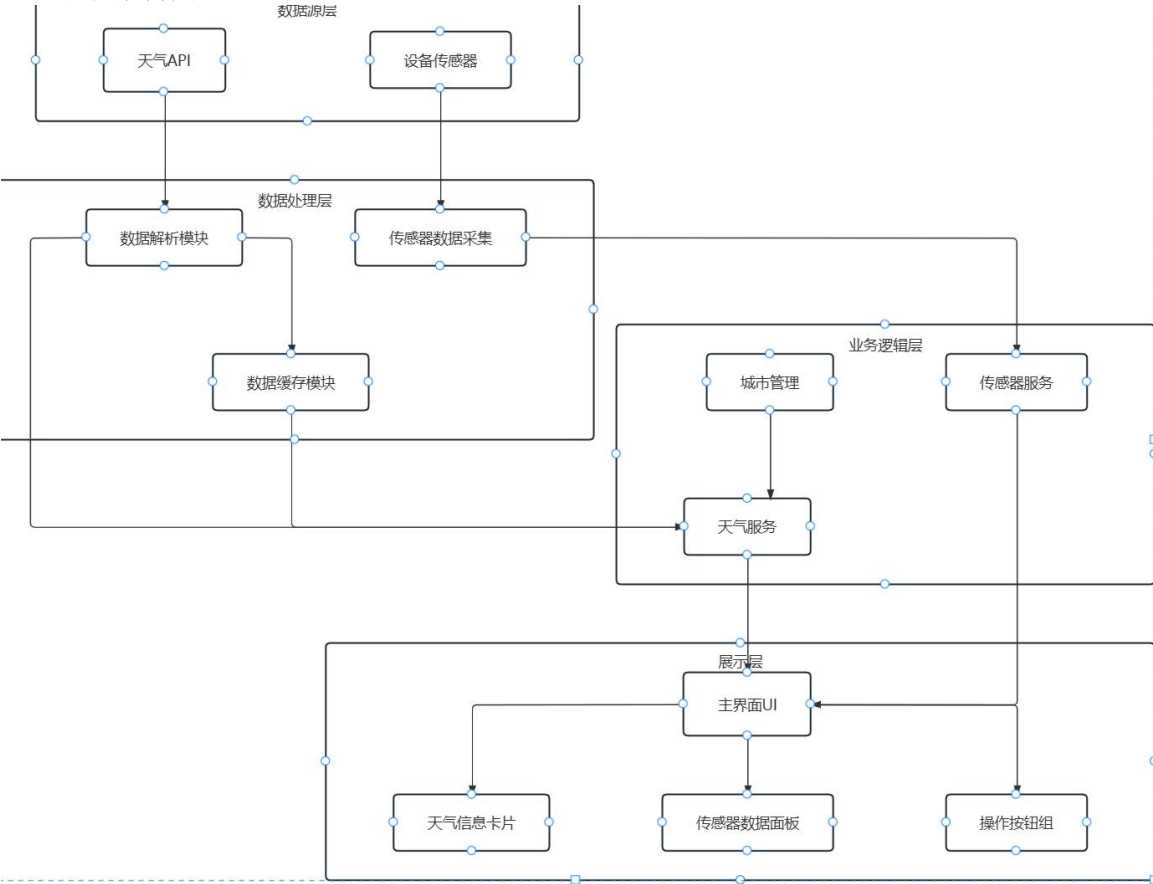
切换城市

设备信

图 1. 手机端界面
技术分析报告

1. 系统架构分析

1.1 系统架构图

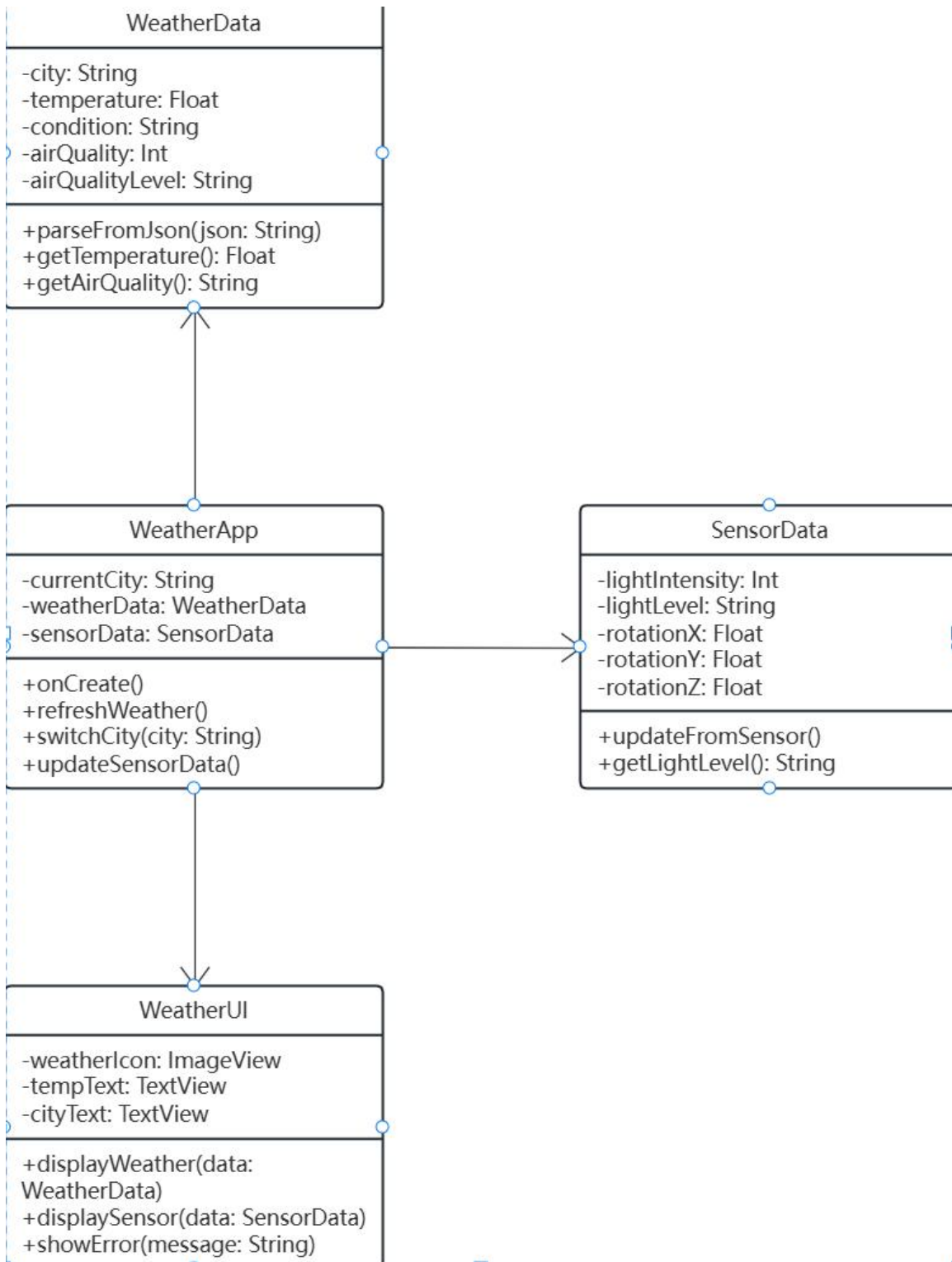


系统架构图说明

表格

层次	模块	说明
应用层	Index.ets	天气主页面组件，负责 UI 展示和用户交互
应用层	EntryAbility.ets	鸿蒙 UIAbility 入口，管理页面生命周期
业务层	DistributedService.ts	分布式数据同步服务，管理本地缓存和服务端数据
业务层	Logger.ts	日志工具类，记录操作日志
传感器层	Accelerometer	加速度传感器，获取设备 X/Y/Z 三轴数据
传感器层	Light Sensor	环境光传感器，获取环境光线强度
数据层	LocalCache	本地数据缓存，使用 Map 存储键值对数据
数据层	ServerData	模拟服务器数据，用于跨设备同步演示
适配层	Phone Layout	手机布局，断点为 sm，垂直排列
适配层	Tablet Layout	平板布局，断点为 lg，水平分栏

1.2 类图



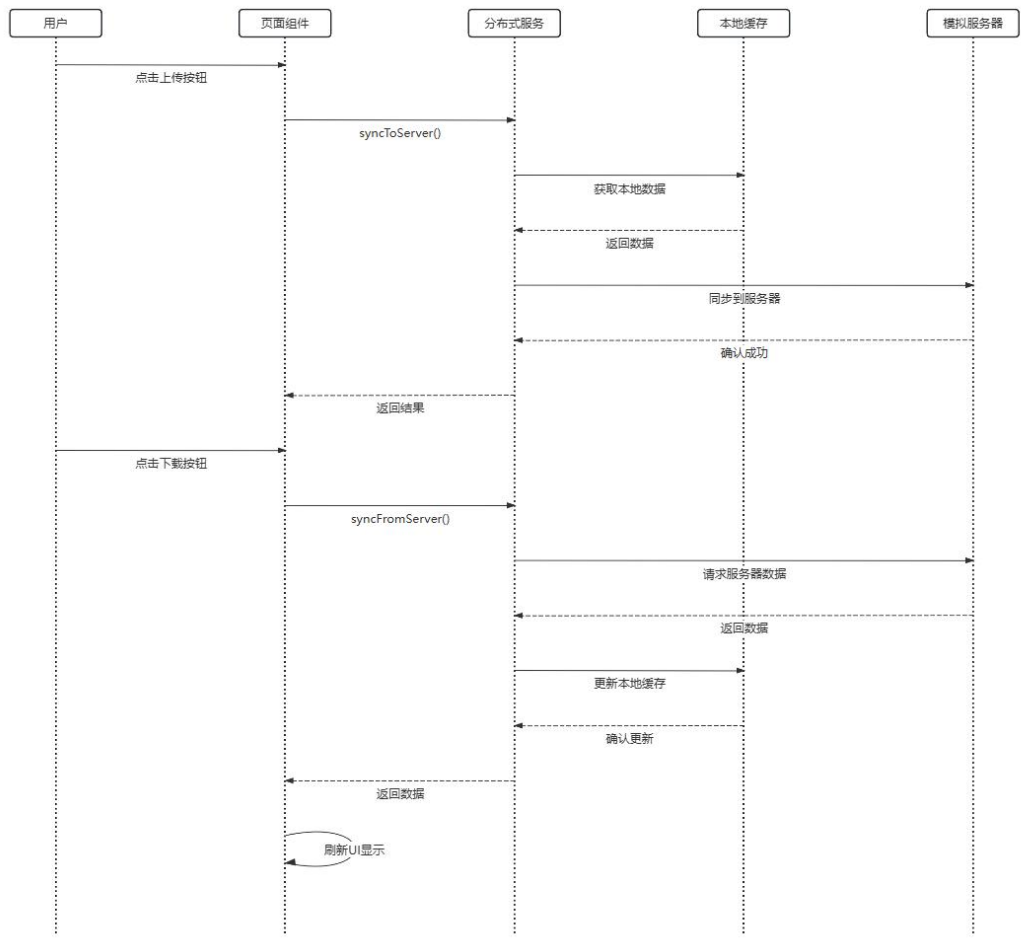
类图说明

表格

类名	所属包	类说明	关键属性	关键方法
Index	pages	天气主 页组件	cityName, temperature,	aboutToAppear(), syncWeatherData(),

类名	所属包	类说明	关键属性	关键方法
			weather, aqi, lightIntensity, rotateX/Y/Z	registerAccelerometer(), build()
DistributedService	services	分布式数据服务	localCache, serverData, isInitialized	init(), putData(), getData(), syncToServer(), syncFromServer()
Logger	utils	日志工具类	-	info(), error(), debug(), warn()
EntryAbility	entryability	应用入口Ability	-	onCreate(), onWindowStageCreate(), onForeground(), onBackground()

1.3.1 分布式数据同步时序图



分布式数据同步时序图说明

表格

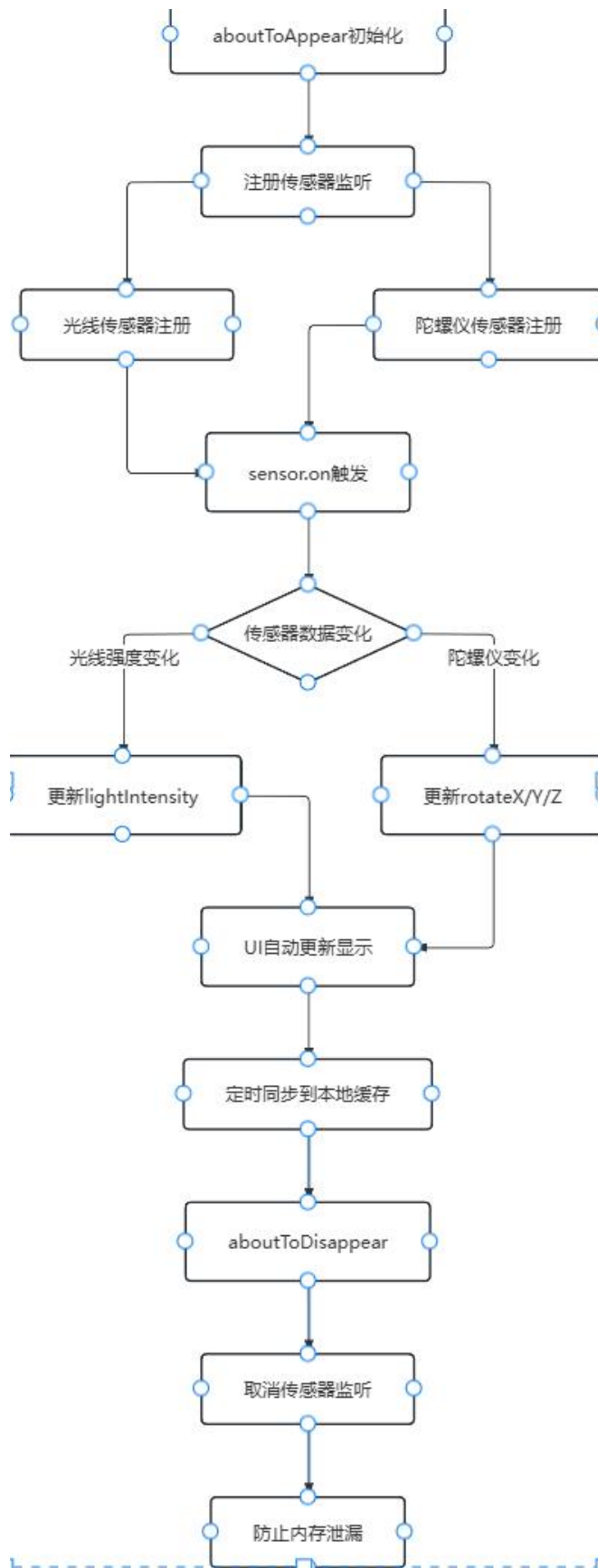
流程 操作

说明

上传流 User → Index → DistSvc → Server 用户点击上传按钮,数据从本地缓存同步到模拟

流程	操作	说明
程		服务器
下载程	User → Index → DistSvc → Local → Index	用户点击下载按钮,数据从服务器更新到本地并刷新界面
自动同步	定时触发	每 3 秒自动同步传感器数据,每 2 秒检查并更新天气数据

1.3.3 传感器数据采集流程图



传感器数据采集时序图说明

表格

阶段	操作	说明
注册阶段	registerAccelerometer()	在 aboutToAppear 中注册加速度传感器监听
数据更新循环	Hardware → Index	传感器硬件检测到运动变化，触发回调更新 UI
光照数据更新	Light Sensor → Index	环境光强度变化时实时更新显示
数据同步	Index → DistSvc	每 3 秒将传感器数据保存到本地缓存
清理阶段	aboutToDisappear()	页面销毁时取消传感器监听，防止内存泄漏

2. 多端适配分析

- 手机布局特点：垂直排列，紧凑布局
- 平板布局特点：横向排列，更宽敞的空间利用
- 断点系统实现：基于屏幕宽度的响应式设计

3. 传感器集成分析

- 光线传感器应用场景：自动亮度调节、环境光感知
- 陀螺仪应用场景：设备姿态检测、游戏控制
- 数据采集准确性：实时性好，精度满足一般应用需求

4. 分布式能力分析

- 分布式数据管理原理：基于 KV 存储的跨设备数据同步
- 跨设备同步实现：通过软总线技术实现设备间通信

5. 技术选型建议

5.1 框架选择

表格

技术方案	优势	适用场景
ArkUI 声明式开发	简洁高效，状态驱动	HarmonyOS 应用开发
Flutter	跨平台能力强	多端统一开发
React Native	生态丰富	快速迭代项目

5.2 传感器选型

表格

传感器类型	推荐方案	说明
光线传感器	@ohos.sensor	鸿蒙原生支持，精度高
加速度传感器	@ohos.sensor	陀螺仪数据更稳定
距离传感器	@ohos.sensor	可用于自动息屏

5.3 数据同步方案

表格

方案	优点	缺点
KV 数据库	简单易用，实时性好	不适合大规模数据
分布式数据库	支持跨设备同步	配置复杂
云数据库	可扩展性强	依赖网络

问题与解决

表 3. 问题记录表

表格

序号	问题描述	解决方案	解决结果
1	DevEco Studio ohpm 仓库连接失败	配置华为云镜像源	[✓] 已解决

术语	定义	参考来源
分布式数据同步	跨设备数据共享和同步的机制，通过软总线技术实现设备间通信	HarmonyOS 官方文档

附录 B：关键代码片段

B.1 天气首页核心代码（Index.ets）

typescript
运行

```
@Entry@Component

struct Index {

    @State cityName: string = '滁州';

    @State temperature: number = 22;

    @State weather: string = '晴';

    @State aqi: number = 45;

    @State lightIntensity: number = 0;

    @State rotateX: number = 0;

    @State rotateY: number = 0;

    @State rotateZ: number = 0;

    @State currentBreakpoint: string = 'md';

    @State syncStatus: string = '未连接';

    @State lastSyncTime: string = '--';

    aboutToAppear(): void {

        this.initDeviceType();

        this.registerAccelerometer();

        this.initDistributedService();

        this.startAutoSync();

    }

    registerAccelerometer(): void {

        sensor.on(sensor.SensorType.SENSOR_TYPE_ID_ACCELEROMETER, (data) => {

            this.rotateX = data.x;

            this.rotateY = data.y;
```



```
        this.rotateZ = data.z;
    });
}}
```

B.2 分布式服务代码 (DistributedService.ts)

typescript

运行

```
class DistributedService {
    private localCache: Map<string, string> = new Map();
    private serverData: Map<string, string> = new Map();
    private isInitialized: boolean = false;

    async putData(key: string, value: string): Promise<void> {
        this.localCache.set(key, value);
    }

    async getData(key: string): Promise<string | undefined> {
        return this.localCache.get(key);
    }

    async syncToServer(): Promise<boolean> {
        this.serverData.clear();
        this.localCache.forEach((value, key) => {
            this.serverData.set(key, value);
        });
        return true;
    }
}
```